ConsultIQ

Growth Compass

Consultant Skill Growth Advisor & CM Team Skill Demand

Technical Design & Implementation Document
ConsultIQ — COS301 Capstone, Team TROOS, in partnership with BBD Software
Demo Wow Factor 2 of 2

1. Overview & Purpose

Growth Compass is one shared simulation engine serving two different audiences, each with their own endpoint, their own scope boundary, and their own framing of the same underlying idea — speculatively adding a skill to a consultant's profile and observing the effect on their fit against real, current project demand.

- **Consultant-facing ("Growth Compass"):** "Which single skill would most improve my standing against the active pipeline, and what would it actually get me?"
- **Consultant-Manager-facing ("Team Skill Demand"):** "Which skills is my team missing relative to what the active pipeline needs, and who specifically needs training?"

Both features share the same scoring core used by real, persisted match runs and by the Project Feasibility Advisor (covered in the companion document) — the same architectural principle applied twice: a speculative, unpersisted computation must never be allowed to diverge from what the real system would actually do.

2. Problem Statement

Consultants on the bench have no structured way to answer *"why am I not getting placed, and what should I learn?"* They can see their current skills and, separately, project postings, but nothing connects the two into an actionable recommendation.

Consultant Managers have no structured view of their own team's skill coverage against *current, real demand* — only ad hoc knowledge of who has what skill, and no connection at all to which of those skills the active pipeline is actually asking for.

3. The Shared Simulation Engine

Two pure, side-effect-free building blocks sit underneath both features:

- `injectHypotheticalSkill()` — given a consultant's current skill set and one candidate skill, returns a *new*, cloned skill set with that one skill added or replaced. Never mutates the original consultant data, and never touches the database.
- `deriveCandidateSkills()` — given a consultant's current skills and the active pipeline's requirements, returns the set of missing **mandatory** skills worth testing, each paired with the most commonly-requested ("modal") competency level for that skill across the pipeline.

Exactly as with the Feasibility Advisor, both `simulate()` (single skill, single project) and `recommendSkillGrowth()` (full batch) route their actual scoring through the shared `MatchScoringExecutorService.scorePool()` — via a reusable `resolveProjectScoringContext()` / `scoreConsultantOnProject()` pair extracted from the original `simulate()` method during a dedicated, isolated refactor, verified before anything was built on top of it.

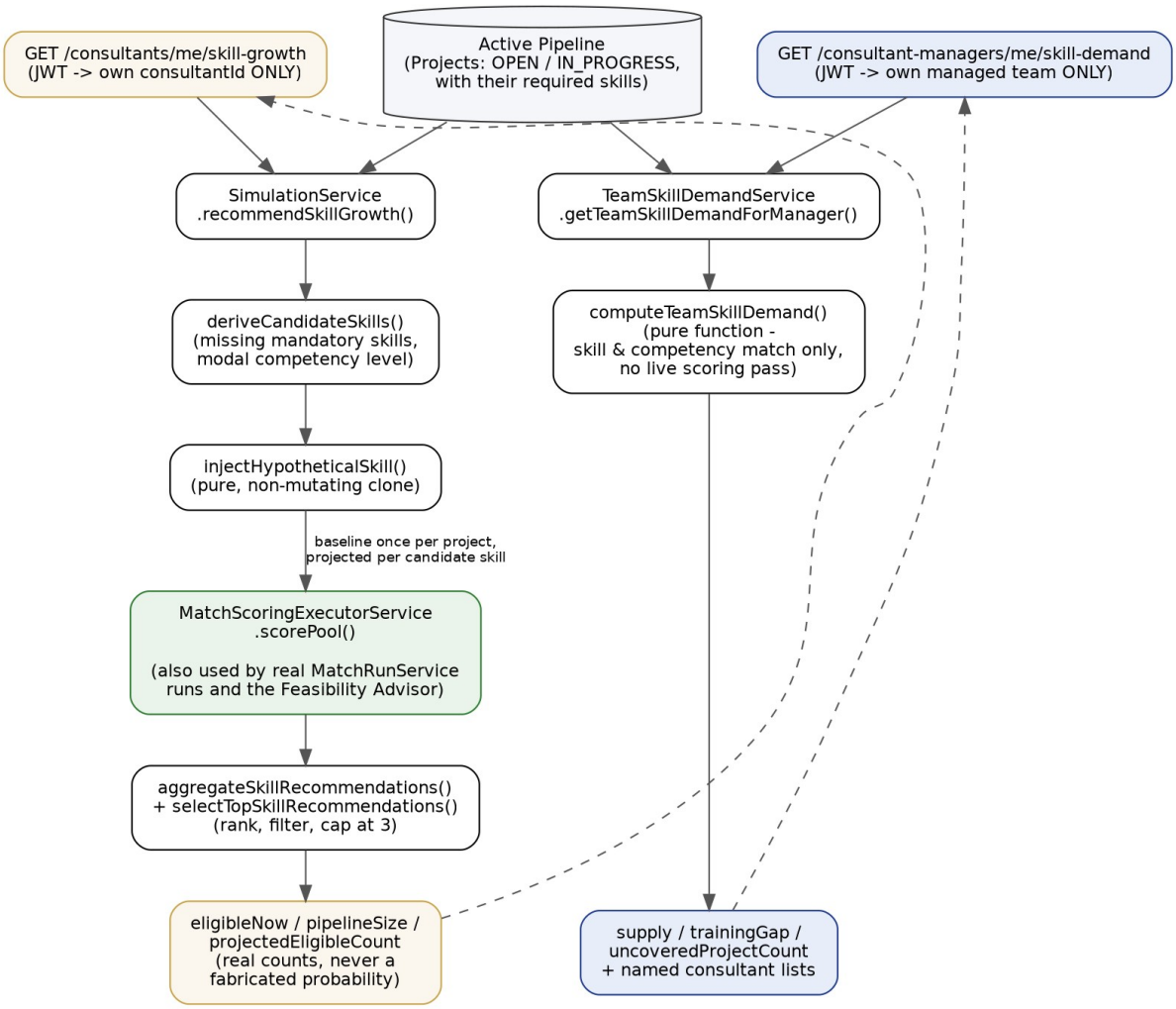


Figure 1 — Two independently-scoped consumers of the active pipeline and the shared scoring core. The CM-facing path performs skill-and-competency matching only — it never issues a live scoring pass.

4. The Baseline-Once Optimization

A naive implementation — testing N candidate skills against M pipeline projects by scoring a baseline and a projected pass for every (skill, project) pair — would require **2 × N × M** scoring passes.

> **Key insight**
>
> A consultant's BASELINE score on a given project — their fit with no hypothetical skill added — never changes across different candidate skills. It is the same consultant, the same project, no hypothetical change. Recomputing it once per candidate skill is pure waste.

The shipped design computes each pipeline project's baseline exactly once, up front, and only varies the projected pass per candidate skill — and even then, only against the subset of pipeline projects that actually mention that skill, rather than the full project set.

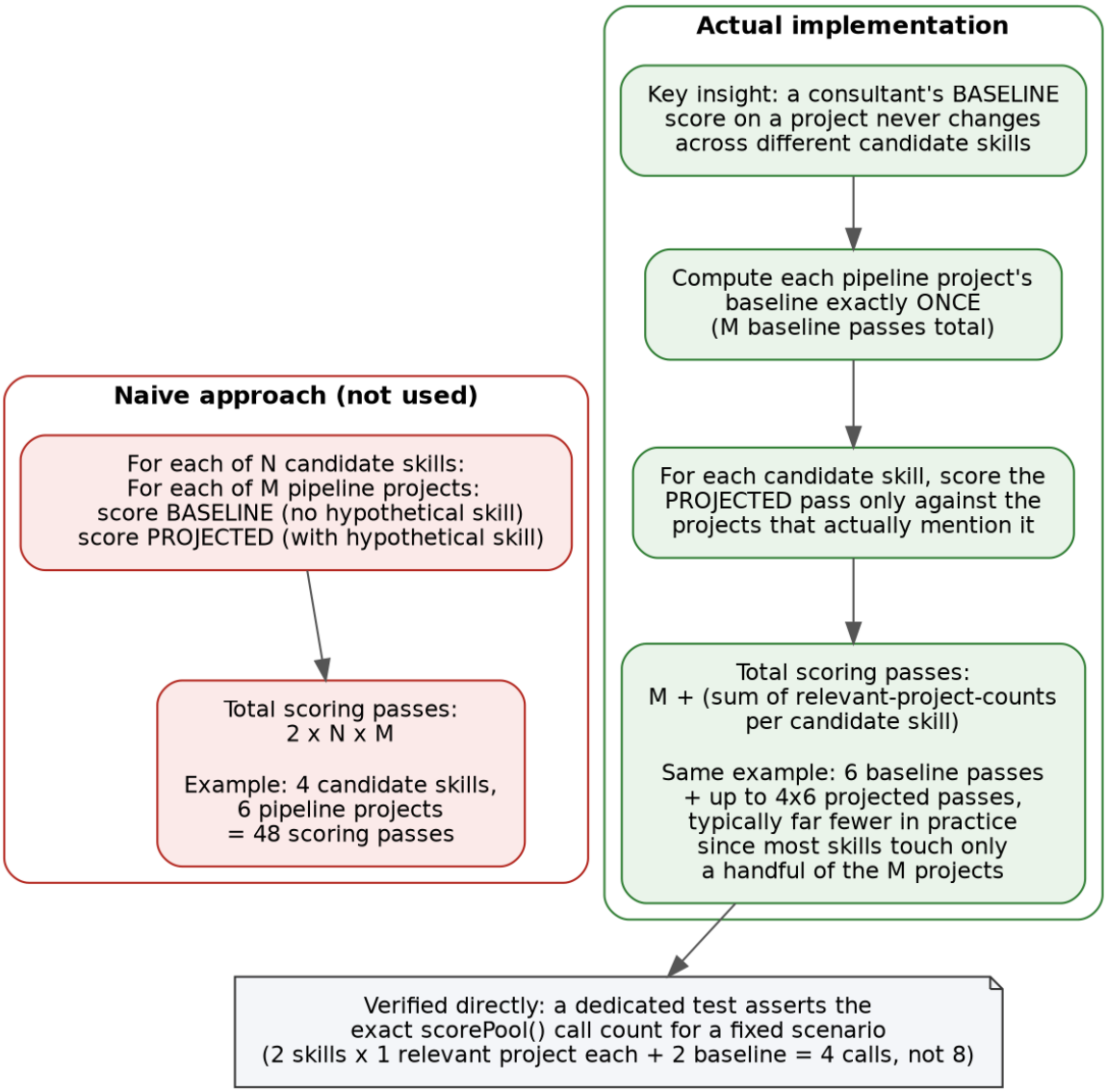


Figure 2 — Naive vs. optimised scoring-pass count for the same batch recommendation request.

This was verified directly, not just reasoned about: a dedicated unit test asserts the exact number of `scorePool()` calls for a fixed scenario (2 candidate skills, 2 pipeline projects, 1 relevant project each) — 4 calls, not 8 — proving the optimisation actually holds in the implementation, not only on paper.

5. Candidate Skill Derivation & Testing Scope

Candidates are the pipeline's mandatory skill requirements the consultant does not already have. Optional (non-mandatory) requirements are deliberately excluded from candidacy — they do not block eligibility, so testing them would not answer the question this feature exists to answer.

Once a skill qualifies as a candidate (via some project's mandatory requirement), it is tested against *every* pipeline project that mentions it at all — mandatory or merely optional on that particular project. Seeing a candidate skill's effect on an optional-requirement project is still genuinely useful information for the consultant; it simply isn't what made the skill worth testing in the first place.

Competency level to test at: the **modal** (most commonly requested) level for that skill across the pipeline — a deliberate simplification over computing the strict minimum viable level, chosen for being simple to compute and simple to explain, at an acceptable cost in precision given the project's timeline.

6. Aggregation & Ranking

`aggregateSkillRecommendations()` collapses the raw, per-(project, skill) rows into one entry per skill:

Field	Meaning
`newlyEligibleProjectCount`	Count of tested projects where the consultant moves from excluded to eligible by adding this skill.
`totalScoreDelta`	Sum of per-project score-point improvements across every tested project. An internal ranking signal only — never shown to the user directly (see Section 7).
`projectsTested`	How many active pipeline projects mention this skill at all.

Ranking order: `newlyEligibleProjectCount` first, `totalScoreDelta` second, skill name alphabetically as a final, deterministic tie-breaker.

`selectTopSkillRecommendations()` then filters out any skill that would help nothing at all (no unlocks and no score gain) **before** capping the list at three — filtering first means a genuinely useless skill can never occupy a slot a further-down, actually-helpful skill should have had.

7. The Eligibility-Framing Correction — a Design Decision Worth Documenting

This section documents a mistake that was caught and corrected during development. The correction itself encodes a design principle that governs every number this feature shows a user, so it is recorded here rather than silently fixed and forgotten.

What shipped first, and was wrong

An early version of the consultant-facing card included a stat tile labelled “Placement likelihood,” displaying totalScoreDelta with a ‘%’ unit and the caption “higher chance of being placed.”

> **Why this was wrong**
>
> totalScoreDelta is a raw SUM of per-project score-point differences — it was never a percentage, and nothing in the scoring engine computes an actual, calibrated placement probability. A real probability would require historical placement-outcome data (who was eligible, who was actually chosen), which this feature never touches anywhere.
>
> The figure could not survive the question “how was this percentage calculated?” — and a number that cannot survive that question does not belong in front of a user making a career decision from it.

The fix that shipped

A real, honestly-computed before/after comparison: `eligibleNow` (how many of the *entire* active pipeline this consultant currently qualifies for) and `projectedEligibleCount` (`eligibleNow` plus the recommended skill’s own `newlyEligibleProjectCount`) — both real project counts out of a real pipeline size. An optional, honestly-labelled percentage (“share of current demand you qualify for”) can be computed from these counts; it is never framed as a probability.

A second, related gap surfaced by this correction: when a skill’s `newlyEligibleProjectCount` is zero — the consultant is already eligible for every project needing it, and the only real benefit is a stronger fit score on projects already won — showing “Today: 8 of 9” beside “With X: 8 of 9” reads as broken, even though the numbers are technically correct. This is handled with a conditional fallback: when there is no new eligibility to show, the card instead surfaces the real average fit-score improvement across the relevant projects, explicitly labelled as a score, never as a probability.

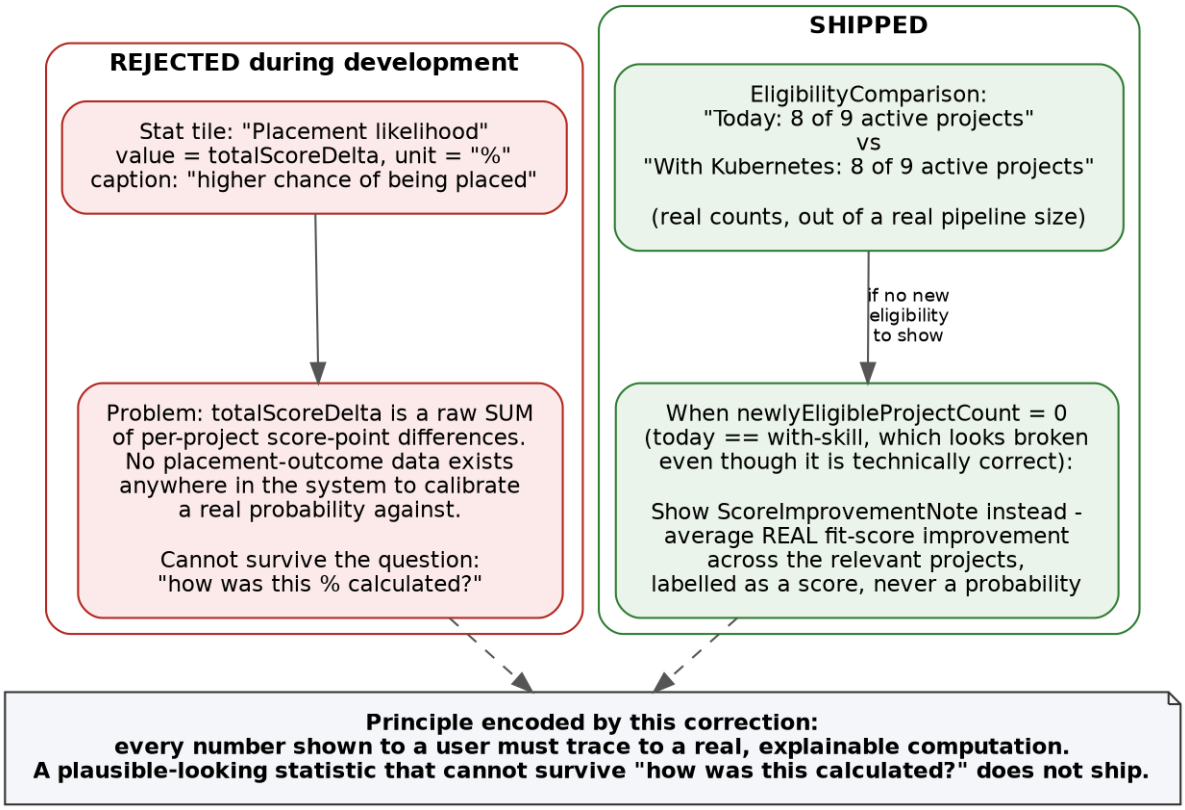


Figure 3 — The rejected framing versus what actually shipped, and the principle the correction encodes.

8. API Contract — Consultant-Facing Endpoint

Route: `GET /consultants/me/skill-growth`

> **Identity resolution**
>
> The consultant identity is taken ONLY from the JWT and resolved server-side to the caller's own consultantId via a minimal lookup (Consultant.findUnique({ where: { userId }, select: { id: true } })). There is no field anywhere in the request through which a consultant could ask for another consultant's recommendations — the boundary exists by construction, not by a runtime check that could be bypassed.

Field	Type	Description
`eligibleNow`	number	How many of the entire active pipeline this consultant currently qualifies for.
`pipelineSize`	number	Total active (OPEN / IN_PROGRESS) pipeline project count.
`recommendations[]`	array	Up to 3 ranked skill recommendations (Section 6).
`recommendations[].projectedEligibleCount`	number	eligibleNow + this skill's own newlyEligibleProjectCount.

Rate limiting: 5 requests per minute per user — stricter than the Feasibility Advisor's 10/minute, because every call here performs meaningfully more scoring work (one baseline pass per pipeline project, plus one projected pass per relevant project per candidate skill). Caching: Redis, per user, 120-second TTL — longer than the Feasibility Advisor's 20 seconds since this is a dashboard card loaded on page visit rather than a live-dragged slider, while still refreshing within about two minutes of a profile change.

9. CM-Facing Extension: Team Skill Demand

9.1 — Scope decision

Explicitly scoped to the requesting CM's **own managed team** (via the `ConsultantManager` join table), never the whole firm. "3 of my 8 consultants need training in AWS" is an actionable coaching decision one CM can act on directly; a firm-wide figure is a statistic no single CM could act on alone.

9.2 — Why this was not built as an extension of the existing SkillGapService

A pre-existing `SkillGapService` already answers a related but genuinely different question — firm-wide portfolio staffing risk ("do we have enough people, across the whole firm, to fully staff this project's team size") — with its own coverage-percentage semantics, at firm-wide scope.

> **Why reuse was rejected**
>
> Extending or repurposing that service for a differently-scoped, differently-defined question risked exactly the statistic-collision problem the Section 7 correction was written to prevent — two different numbers both claiming to answer "does the team have skill X," disagreeing with each other, with no way for a CM to know which one to trust. A new, narrowly-scoped service was built instead.

9.3 — The three stats, precisely defined

Stat	Definition
`supply`	Count of this CM's consultants with the skill at ANY competency level — a flat headcount, not competency-aware, matching how the requirement was originally phrased ("how many consultants have this skill").
`trainingGap`	teamSize − supply.
`uncoveredProjectCount`	Of the active pipeline projects requiring the skill, how many have ZERO of this CM's consultants meeting that project's SPECIFIC required competency level. This one IS competency-aware — unlike supply, "can we actually staff this" genuinely depends on the level required.

> **Explicit caveat, documented in the service itself**
>
> uncoveredProjectCount checks skill-and-competency coverage only. It does not factor in budget, location, or availability exclusions the way the real scoring engine would. "Covered" here means "someone has the right skill at the right level," not "this project is definitely staffable."

9.4 — Named consultant lists: a deliberate, narrower privacy boundary than the consultant side

Each skill entry also carries `consultantsWithSkill` and `consultantsNeedingTraining` — named lists, not just counts. This is a deliberate departure from the consultant-facing feature (Section 8), where a consultant can never see another consultant's data. It is not a privacy regression: CMs are already permitted, elsewhere in the system, to see their own team members by name (the same boundary that governs the rest of the application — CMs see their own consultants, PMs see all consultants but only their own projects). Naming consultants here stays within that existing boundary; it does not extend it.

The two count fields are **derived from** the lengths of the two named arrays, not computed independently — `supply === consultantsWithSkill.length` by construction, so the counts and the names can never silently drift apart.

10. API Contract — CM-Facing Endpoint

Route: `GET /consultant-managers/me/skill-demand`

Field	Type	Description
`teamSize`	number	Total consultants this CM manages (all statuses, not just currently bookable).
`skills[]`	array	One entry per skill mandatorily required somewhere in the active pipeline, sorted most-urgent-first (uncoveredProjectCount desc, trainingGap desc, name asc).
`skills[].consultantsWithSkill`	ConsultantSummary[]	consultantId, fullName, email — scoped strictly to this CM's own team.
`skills[].consultantsNeedingTraining`	ConsultantSummary[]	Same shape; the complement set.

No dedicated cache or throttler was built for this endpoint — it performs plain, inexpensive counting queries with no live scoring pass, well within the cost the application's existing global default rate limit was already designed to absorb.

11. Testing Strategy

Layer	Coverage
Unit — pure functions	injectHypotheticalSkill, deriveCandidateSkills, aggregateSkillRecommendations, selectTopSkillRecommendations, computeTeamSkillDemand — all tested with plain in-memory fixtures, no mocking required.
Unit — SimulationService	Baseline/projected pool isolation (the hypothetical skill never leaks into the baseline pool); the exact scorePool() call-count optimisation (Section 4); exclusion-flag handling on both sides of a simulation.
Unit — TeamSkillDemandService	Manager-to-consultant-id resolution; pipeline scoped to OPEN/IN_PROGRESS only; a manager with zero assigned consultants throws cleanly rather than returning an empty-but-misleading result.
Integration (real test DB)	Zero persistence across single and repeated calls; two consultants with different skill sets receive correctly isolated, non-overlapping recommendations; a consultant's baseline score matches a real, executed match run for an identical input.

12. Frontend: the Growth Compass Card

Two purpose-built components carry the Section 7 correction into the UI: `EligibilityComparison` (the real today-vs-with-skill counts) and `ScoreImprovementNote` (the honest fallback when there is no new eligibility to show). The card renders whichever is appropriate based on the top recommendation's `newlyEligibleProjectCount`.

As with the Feasibility Advisor's frontend, the recommendation list is rendered **in the exact order the backend returns it** — no client-side re-filtering or re-sorting is applied, so the frontend can never silently contradict the backend's own, already-correct ranking decision.

13. Known Limitations & Future Work

- The modal (rather than strict minimum-viable) competency-level simplification trades some precision for simplicity — a future iteration could compute the true minimum level needed to flip each specific project.
- Scoped entirely to the current active pipeline; there is no longer-range "best skill for your whole career trajectory" view.
- uncoveredProjectCount is skill-and-competency coverage only (Section 9.3); a fuller version would run the complete scoring-engine eligibility check per consultant per project, at meaningfully higher compute cost — deferred given the project timeline.